

实验报告 — SVM 分类与回归 (task4)

实验目的

1. 使用支持向量分类器 (SVC) 在银行票据认证数据集上比较不同核 (linear、rbf、poly、sigmoid) 及自定义核的分类效果，并可视化决策边界。
2. 使用支持向量回归 (SVR) 在白葡萄酒质量数据集上比较不同核的回归性能 (计算 MSE)，并尝试自定义核。
3. 扩展实验：对银行票据数据集使用更多特征 (而非单对特征)，绘制每对特征的决策边界，使用 10 折交叉验证选择最佳核，并计算 Precision/Recall/F1 等指标。

实验环境

- 操作系统：Windows (在用户环境的 Conda Python 3.10 上验证)
- Python 版本：建议 3.8+ (已在 3.10 环境测试)
- 关键依赖：numpy、pandas、matplotlib、scikit-learn (详见 [task4/requirements.txt](#))
- 工作目录：`c:\Users\kevinxin\Desktop\machine_learning\task4`

实验过程 (概要)

1. 数据准备

- 银行票据数据集：`data_banknote_authentication.txt` (4 个特征 + class)。从原始文件读入，先随机抽取 20% 作为工作集，然后再将工作集按 80%/20% 划分为训练/测试。
- 葡萄酒数据集：`winequality-white.csv` (含表头，最后一列 `quality` 为目标)。同样随机抽取 20% 作为工作集，再按 80%/20% 划分训练/测试。对特征使用 `StandardScaler` 进行标准化后再训练 SVR。

2. SVC 实验 (`svc-banknote.py`)

- 选择若干特征对 (如 (0,1)、(0,2)、(1,2)) 进行二特征可视化实验。
- 分别训练内置核 (linear、rbf、poly、sigmoid)，绘制训练数据点与决策边界。
- 实现并测试自定义核 (cubic、poly3、gaussian)，并示范 Gram 矩阵 (precomputed kernel) 用法 (注意：precomputed 不直接支持独立测试集的普通预测，需要额外构造 `test×train` 的核矩阵)。

3. SVR 实验 (`svr-winequality.py`)

- 对特征做 `StandardScaler`。
- 使用四种内置核 (linear、rbf、poly、sigmoid) 训练 SVR 并计算测试集上的均方误差 (MSE)。
- 测试自定义核 (cubic、poly3、gaussian)，并保存对应的预测 vs 真实图。

4. 扩展任务 (`svc-more-banknote.py`)

- 使用数据中的所有特征，两两配对绘制决策边界 (保存为图片)。
- 使用 10 折交叉验证 (StratifiedKFold) 在训练集上比较核函数的 F1 分数以选出最佳核。
- 在测试集上使用选出的最佳核评估并输出 Accuracy / Precision / Recall / F1 (保存到 `outputs/metrics.txt`)。

测试样例数据说明

- `data_banknote_authentication.txt`: 每行含 4 个数值特征 (variance, skewness, curtosis, entropy) 和一个 class (0/1) 。
- `winequality-white.csv`: 含表头的 CSV 文件, 字段示例: "fixed acidity","volatile acidity",..., "alcohol","quality"。目标为 `quality` (整数评分) 。

测试结果 (按你要求的 todo 顺序)

下面的结果按我们之前的 todo 顺序组织 (创建脚本 -> 运行并生成输出 -> 验证并生成报告 -> 总结/下一步) 。

1. 创建脚本 (结果文件/说明)

- 已创建并保存在 `task4/`:
 - `svc-banknote.py` (SVC 实验)
 - `svr-winequality.py` (SVR 实验, 包含 `StandardScaler`)
 - `svc-more-banknote.py` (扩展实验, 10 折 CV 与多特征对可视化)
 - `compute_mse.py`, `compute_custom.py` (用于批量计算 MSE, 辅助复现实验)
 - `requirements.txt`、`README.md` (实验说明)

2. 安装依赖并运行脚本 (生成的主要输出文件与说明)

- 运行命令 (示例) :

```
cd c:\Users\kevinxin\Desktop\machine_learning\task4
pip install -r requirements.txt
python svc-banknote.py
python svr-winequality.py
python svc-more-banknote.py
```

- 脚本运行后输出位于 `task4/outputs/`, 包含:
 - SVC 决策边界图 (示例) :
 - `svc_0_1_linear.png`, `svc_0_1_rbf.png`, `svc_0_1_poly.png`, `svc_0_1_sigmoid.png`
 - `svc_0_2_*.png`, `svc_1_2_*.png` (其它特征对)
 - 自定义核图: `svc_custom_cubic_0_1.png`, `svc_custom_poly3_0_1.png`, `svc_custom_gaussian_0_1.png`
 - Gram 矩阵示例: `svc_precomputed_cubic.png`
 - SVR 预测 vs 真实图: `svr_linear.png`, `svr_rbf.png`, `svr_poly.png`, `svr_sigmoid.png`
 - SVR 自定义核图: `svr_custom_cubic.png`, `svr_custom_poly3.png`, `svr_custom_gaussian.png`
 - 扩展任务决策边界: `pair_0_1_rbf.png`, `pair_0_2_rbf.png`, `pair_0_3_rbf.png`, `pair_1_2_rbf.png`, `pair_1_3_rbf.png`, `pair_2_3_rbf.png`
 - 评估文本: `metrics.txt` (包含 10 折 CV 选出的最佳核以及 Accuracy/Precision/Recall/F1)

3. 验证并生成报告 (我已把结果嵌入到 `README.md`, 并额外生成了 `readmenew.md`)

- 我已把关键图片路径与已计算的数值写入最终报告 (`readmenew.md`) , 供你直接查看或拷贝到汇报中。

4. 汇总/总结 (见下)

我建议你在报告中使用的关键图片 (按模块对应)

- SVC (银行票据, 选择一组典型特征对用于展示) :
 - `outputs/svc_0_1_rbf.png` (rbf 的典型决策边界)
 - `outputs/svc_0_1_linear.png` (linear)
 - `outputs/svc_custom_cubic_0_1.png` (自定义立方核示例)
 - `outputs/svc_precomputed_cubic.png` (Gram 矩阵示例)
- SVR (葡萄酒, 预测 vs 真实) :
 - `outputs/svr_rbf.png` (rbf, 通常效果较好)
 - `outputs/svr_linear.png` (linear, 作为对比)
 - `outputs/svr_poly.png` (poly, 注意多项式可能过拟合)
 - 如果需要展示自定义核效果: `outputs/svr_custom_cubic.png`, `outputs/svr_custom_gaussian.png`
- 扩展任务 (多特征决策边界与 CV 结果) :
 - `outputs/pair_0_1_rbf.png`, `outputs/pair_0_2_rbf.png`, `outputs/pair_1_2_rbf.png` (展示不同特征对下的决策边界差异)
 - `outputs/metrics.txt` (放在报告附件中或直接贴入报告中作为表格)

测试结果 (关键数值摘录)

- SVMSE:
 - linear: 0.7221
 - rbf: 0.6760
 - poly: 2.8487
 - sigmoid: 37.8879
 - cubic (自定义): 48.8037
- SVC 扩展任务 (来自 `outputs/metrics.txt`, 包含每个核在测试集上的评估) :

```
=== Evaluation on test set for each kernel ===
Test size: 55
Test set class distribution:
1: 29
0: 26

Kernel: linear
Accuracy: 1.0000, Precision: 1.0000, Recall: 1.0000, F1: 1.0000
Confusion matrix:
26 0
0 29

Kernel: rbf
Accuracy: 1.0000, Precision: 1.0000, Recall: 1.0000, F1: 1.0000
```

```
Confusion matrix:
26 0
0 29

Kernel: poly
Accuracy: 0.9273, Precision: 0.8788, Recall: 1.0000, F1: 0.9355
Confusion matrix:
22 4
0 29

Kernel: sigmoid
Accuracy: 0.7273, Precision: 0.7692, Recall: 0.6897, F1: 0.7273
Confusion matrix:
20 6
9 20
```

=== Classification reports (short) ===

Kernel: linear

	precision	recall	f1-score	support
0	1.0000	1.0000	1.0000	26
1	1.0000	1.0000	1.0000	29
accuracy			1.0000	55
macro avg	1.0000	1.0000	1.0000	55
weighted avg	1.0000	1.0000	1.0000	55

Kernel: rbf

	precision	recall	f1-score	support
0	1.0000	1.0000	1.0000	26
1	1.0000	1.0000	1.0000	29
accuracy			1.0000	55
macro avg	1.0000	1.0000	1.0000	55
weighted avg	1.0000	1.0000	1.0000	55

Kernel: poly

	precision	recall	f1-score	support
0	1.0000	0.8462	0.9167	26
1	0.8788	1.0000	0.9355	29
accuracy			0.9273	55
macro avg	0.9394	0.9231	0.9261	55
weighted avg	0.9361	0.9273	0.9266	55

Kernel: sigmoid

	precision	recall	f1-score	support
0	0.6897	0.7692	0.7273	26
1	0.7692	0.6897	0.7273	29

accuracy			0.7273	55
macro avg	0.7294	0.7294	0.7273	55
weighted avg	0.7316	0.7273	0.7273	55

(提示：上面数值来源于当前一次运行；如果你在不同的随机种子或不同的抽样下重复实验，数值可能会改变。)

总结与问题

1. 已达成目标：

- 实现并运行了 SVC（多核与自定义核）的可视化实验，生成了决策边界图。
- 实现并运行了 SVR（包括 StandardScaler 预处理），得到 MSE 评价并保存预测图。
- 完成扩展任务：多特征对可视化、10 折交叉验证与精确率/召回率/F1 的计算。

2. 已知问题与注意事项：

- Gram 矩阵（precomputed kernel）示例已演示，但其直接使用方式与普通 kernel 参数不同：对独立测试集需要构造 test×train 的 Gram 矩阵再预测。
- 部分核（例如 SVR 的 sigmoid、自定义某些核）在默认超参数下表现很差（MSE 很大），建议做特征缩放（已做）、并对 C、gamma、epsilon 等超参数进行网格搜索。
- SVC 在部分运行中给出 Accuracy/Precision/Recall/F1 为 1.0，需警惕可能的过拟合或样本划分偏差，建议重复多次随机划分并取均值/方差来报告更稳健的结果。
- 决策边界图采用网格采样绘制，若数据或特征尺度较大，请适当增大 `plot_step` 以防内存占用过高。

3. 后续建议（可选）：

- 对 SVC/SVR 增加网格搜索（GridSearchCV）或随机搜索（RandomizedSearchCV）以调参并记录最佳参数。
- 在报告中加入混淆矩阵、每类的 precision/recall（SVC）以及残差分布（SVR）的直方图或箱线图。
- 将最终报告导出为 PDF（或包含内嵌图片的 Markdown）便于提交或打印。

文件位置：`c:\Users\kevinxin\Desktop\machine_learning\task4\readmenew.md`

如果你希望我把 `readmenew.md` 的图片改为 Base64 内嵌（使单个文件自包含）或生成 PDF，我可以继续执行这些步骤；如果现在就够了，请确认结束。